

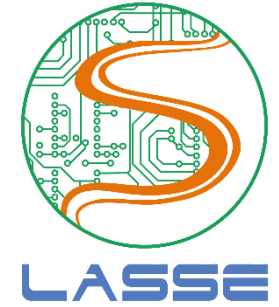
Inteligência Computacional

Introduction to "nearest neighbor" classification



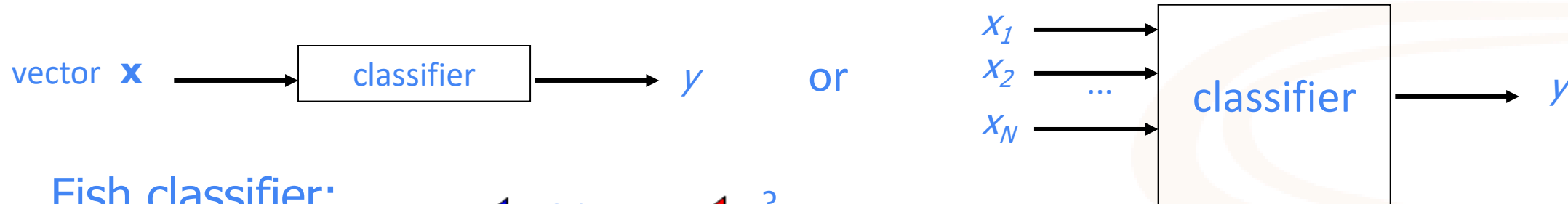
Aldebaro Klautau
Federal University of Para (UFPA)
November 26, 2024

Classification



Problem:

- Given a vector $\mathbf{x}$ with N parameters, find its class y among Y possible labels



Fish classifier:



Training set
used during →
the training stage

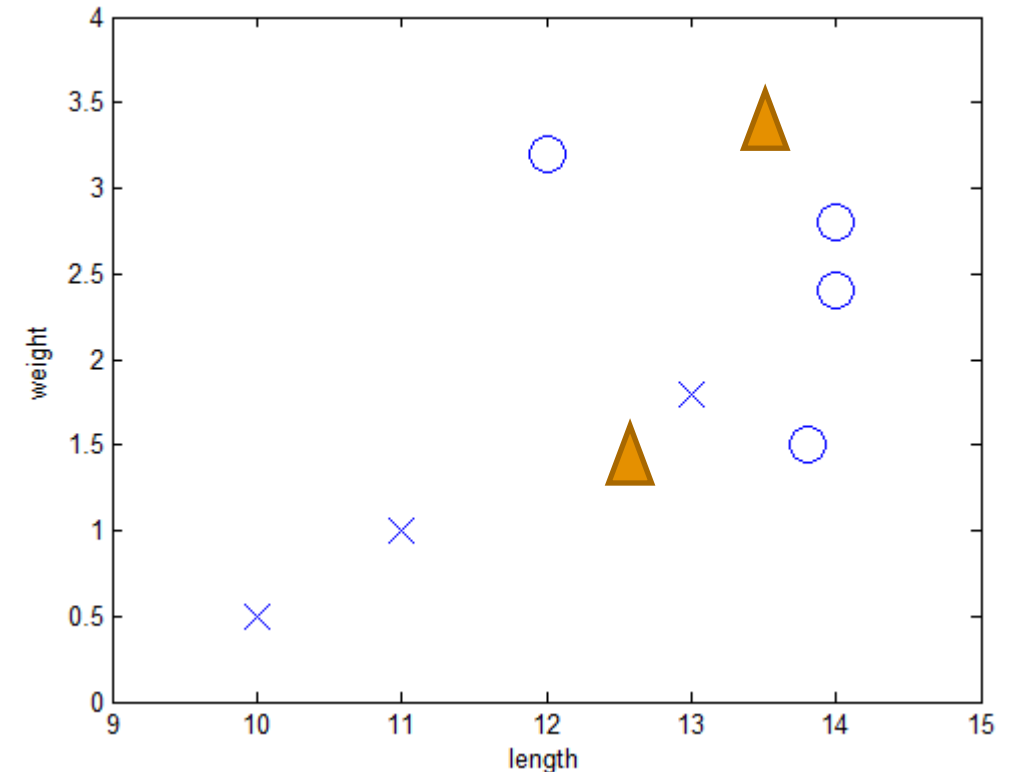
Length	Weight	Class y
12	3.2	
10	0.5	
14	2.8	

Test stage uses “unseen”
examples:
 $\mathbf{x} = (13, 4.2)$
 $y = ?$

Evaluation: misclassification (or error) rate

Nearest neighbor classifier

- The nearest neighbor (NN) classifier simply stores the whole training sequence and, according to the adopted distance measure (e.g. Euclidean distance) assigns to the test example the same class of its nearest (smallest distance) neighbor (example of the stored training sequence)
- The Euclidean distance corresponds to the squared value of the error vector norm
 - y represents the test vector ▲
 - z represents a training example
 - Euclidean: $\text{distance}(y, z) = ||y - z||^2$





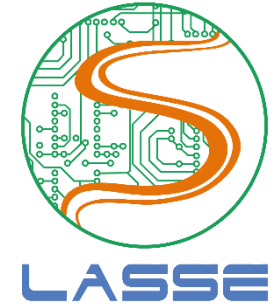
NN classifier implementation

```
1- x_train=load('train_set.txt'); %load from file to memory
2- x_test=load('test_set.txt'); %load from file to memory
3- [N_train,M]=size(x_train); %N_train is the number of examples and M=3
4- [N_test,M]=size(x_test); %N_test is the number of test examples
5- numberOfErrors=0; %count number of errors
6- for i=1:N_test %go over all test set
7-     testFeatures = x_test(i,1:2);%extract only features (discard label)
8-     %find nearest neighbor in training set
9-     minimumDistance = Inf; %initialize with large number
10-    nearestNeighborIndex = -1;
11-    for j=1:N_train %go over all training set
12-        trainFeatures = x_train(j,1:2);%extract only features (discard label)
13-        distance = sum( (testFeatures-trainFeatures).^2 ); %squared norm
14-        if distance < minimumDistance %check if better than current
```

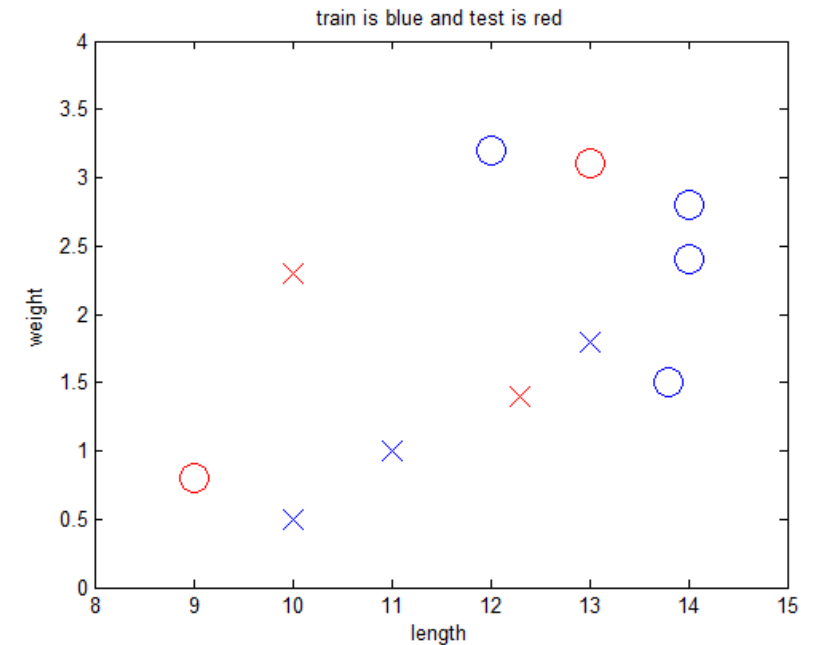
NN classifier implementation

```
12 -     trainFeatures = x_train(j,1:2); %extract only features (discard la
13 -     distance = sum( (testFeatures-trainFeatures).^2 ); %squared norm
14 -     if distance < minimumDistance %check if better than current
15 -         nearestNeighborIndex = j; %store the current NN
16 -         minimumDistance = distance; %update minimum distance
17 -     end
18 - end
19 - classifierOutput = x_train(nearestNeighborIndex,3); %decision
20 - if classifierOutput ~= x_test(i,3) %compare with correct label
21 -     numberOfErrors = numberOfErrors + 1; %update number of errors
22 - end
23 - end
```

NN classifier implementation

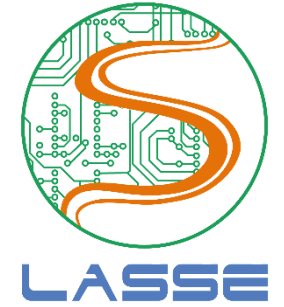


```
1- x_train=load('train_set.txt'); %load from file to memory
2- x_test=load('test_set.txt'); %load from file to memory
3- [N_train,M]=size(x_train); %N_train is the number of examples and M=3
4- [N_test,M]=size(x_test); %N_test is the number of test examples
5- numberOfErrors=0; %count number of errors
6- for i=1:N_test %go over all test set
7-     testFeatures = x_test(i,1:2);%extract only features (discard label)
8-     %find nearest neighbor in training set
9-     minimumDistance = Inf; %initialize with large number
10-    nearestNeighborIndex = -1;
11-    for j=1:N_train %go over all training set
12-        trainFeatures = x_train(j,1:2);%extract only features (discard label)
13-        distance = sum( (testFeatures-trainFeatures).^2 ); %squared distance
14-        if distance < minimumDistance %check if better than current
15-            nearestNeighborIndex = j; %store the current NN
16-            minimumDistance = distance; %update minimum distance
17-        end
18-    end
19-    classifierOutput = x_train(nearestNeighborIndex,3); %decision
20-    if classifierOutput ~= x_test(i,3) %compare with correct label
21-        numberOfErrors = numberOfErrors + 1; %update number of errors
22-    end
23- end
```



Error rate on test set = 25 %
(error on training set is always 0)

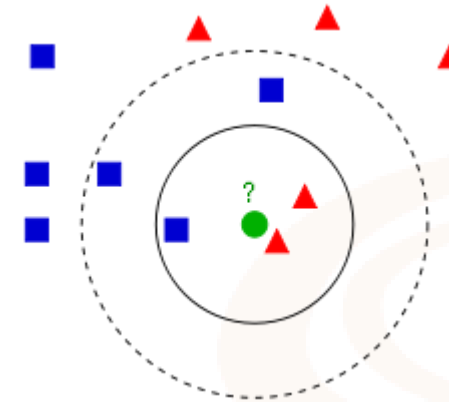
Generalizing the NN: K-nearest neighbors



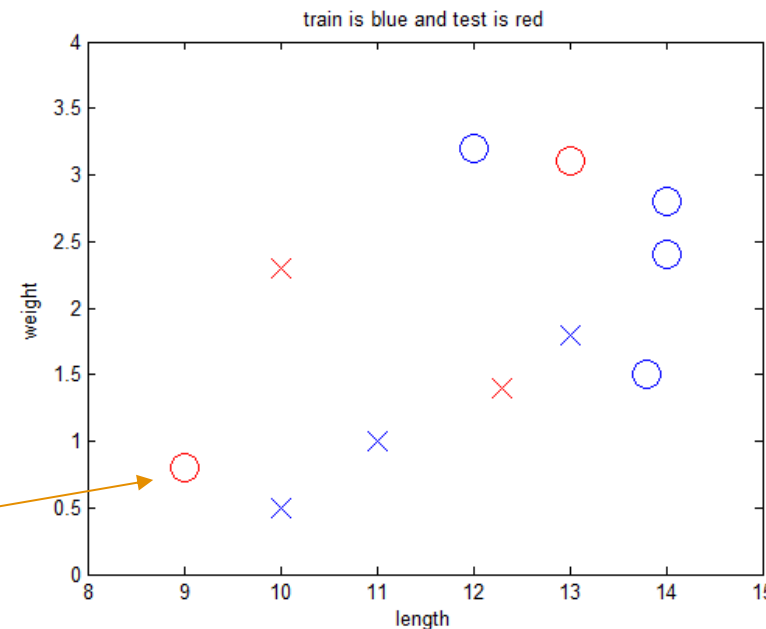
➤ KNN classifier: choose K as an integer odd number (e.g. $K=3$ or 5 is widely adopted) and make the classifier to output the most popular label among the K nearest neighbor as final decision

➤ The previous NN classifier is equivalent to using $K=1$, which may be less robust to *outliers* than $K > 1$

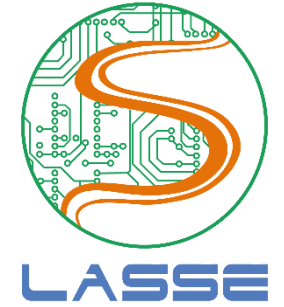
➤ Example: assume that all red and blue examples compose a new training sequence. Note the outlier!



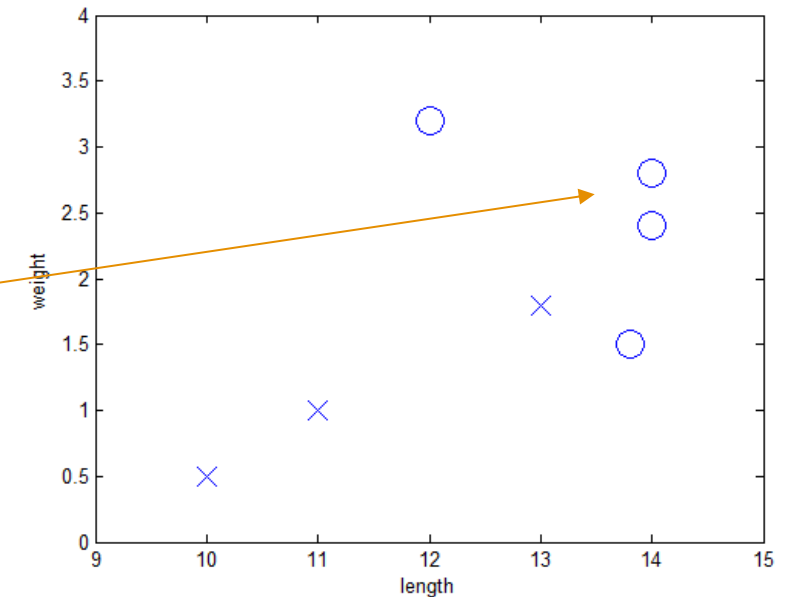
Example from Wikipedia
en.wikipedia.org/wiki/K-nearest_neighbors_algorithm



Controlling number of examples in KNN



- KNN is called “lazy” classifier because “training” corresponds to storing the whole training sequence. But this can lead to a prohibitive computational cost
- To decrease the number of examples stored per class, a “clustering” algorithm such as K-means can be used (the meaning for K in K-means is **not** same as K in K-NN)
 - Example, running K-means with K=2 for the 4 examples of label 0 would return 2 new vectors to represent the class 0
- K-means is an iterative algorithm, but in the special case of K=1 with Euclidean distance, its outputs is simply the average of the input vectors
 - ➔ Adopting K-means with K=1 “mean” (1 example per class), would lead to a simple KNN with K=1 (IB1 in Weka)



Norms

- Besides the Euclidean distance, there are several norms that can be used to compare (error) vectors and matrices. Norms are used to define distances:

$$d(x,y) = ||x-y||$$

- $\text{norm}([3 \ 4],1)=7$

- $\text{norm}([3 \ 4]) = 5$

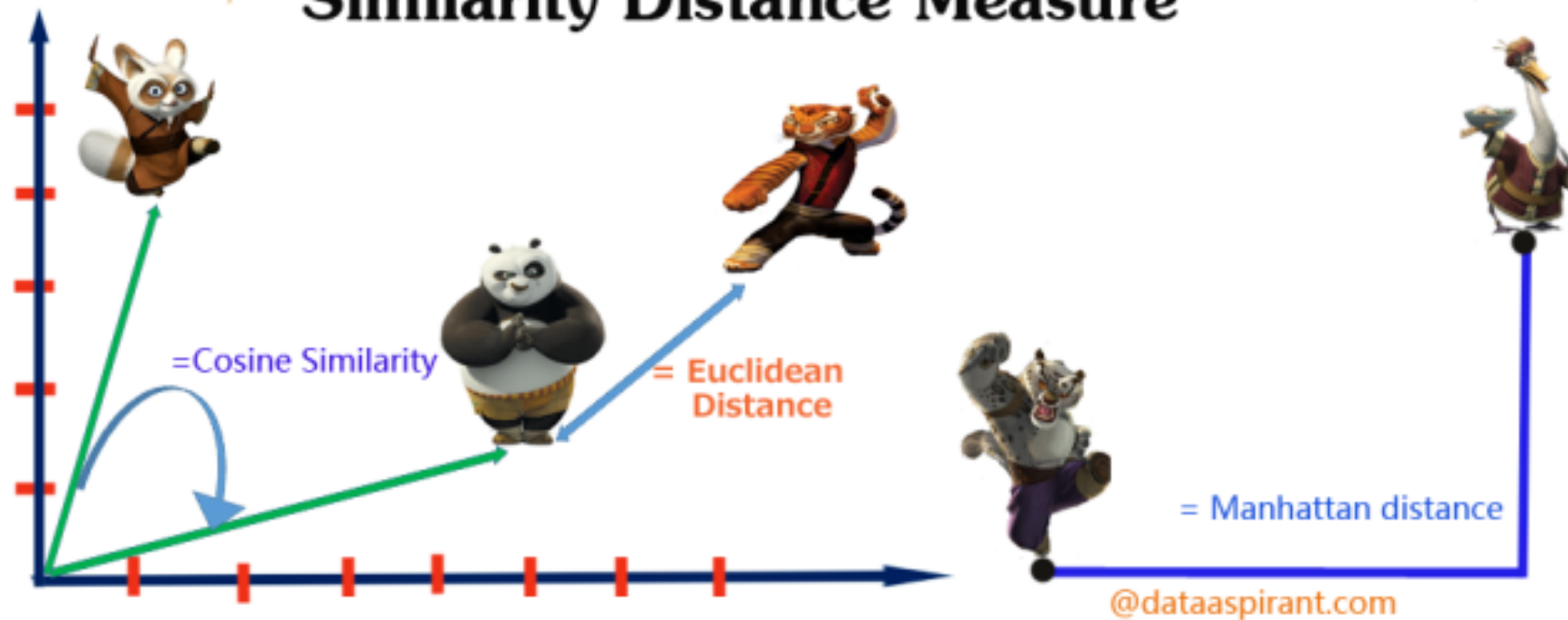
- $\text{norm}([3 \ 4],\text{'fro'})=5$

- $\text{norm}([3 \ 4],\text{Inf}) = 4$
<http://www.netlib.org/lapack/lug/node75.html>

	Vector	Matrix
one-norm	$\ x\ _1 = \sum_i x_i $	$\ A\ _1 = \max_j \sum_i a_{ij} $
two-norm	$\ x\ _2 = (\sum_i x_i ^2)^{1/2}$	$\ A\ _2 = \max_{x \neq 0} \ Ax\ _2 / \ x\ _2$
Frobenius norm	$\ x\ _F = \ x\ _2$	$\ A\ _F = (\sum_{ij} a_{ij} ^2)^{1/2}$
infinity-norm	$\ x\ _\infty = \max_i x_i $	$\ A\ _\infty = \max_i \sum_j a_{ij} $

There are several similarity measures for features organized as vectors

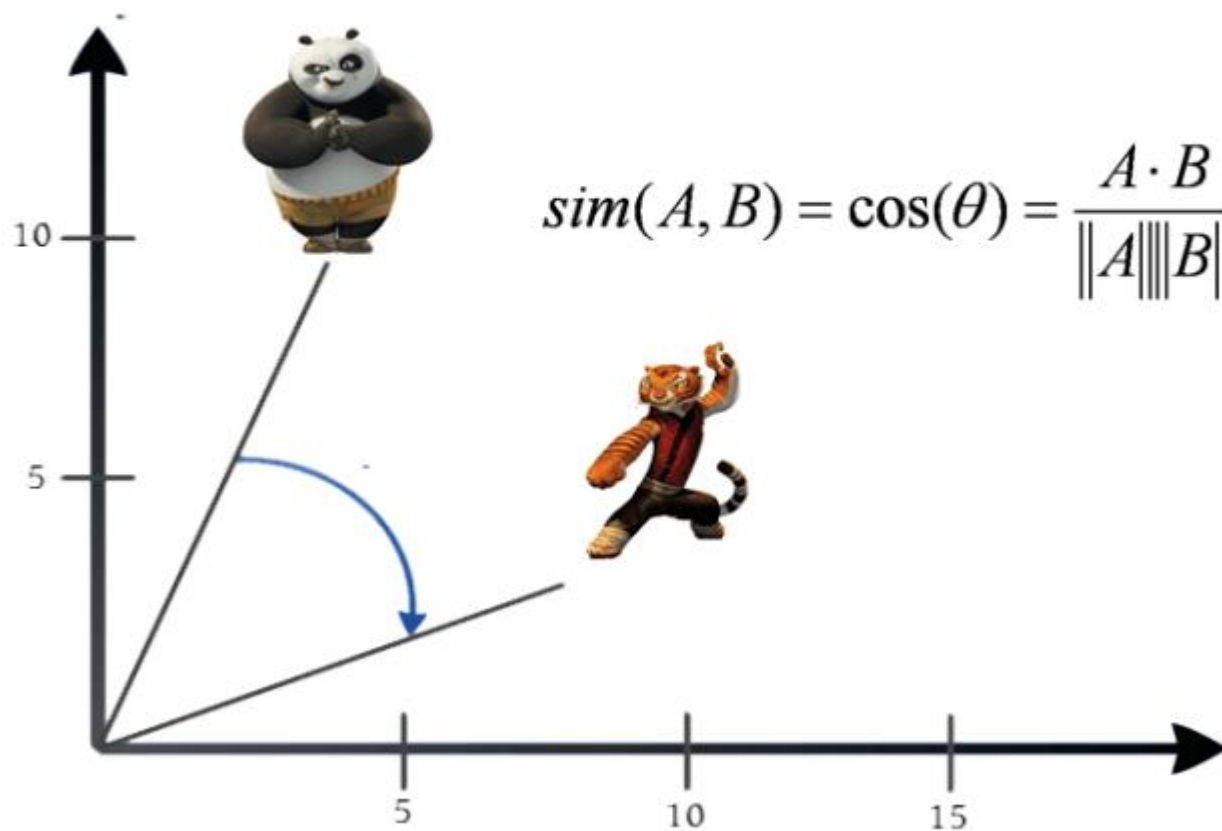
Similarity Distance Measure



↳ <http://dataaspirant.com/2015/04/11/five-most-popular-similarity-measures-implementation-in-python/>

Similarity based on inner product

Cosine Similarity

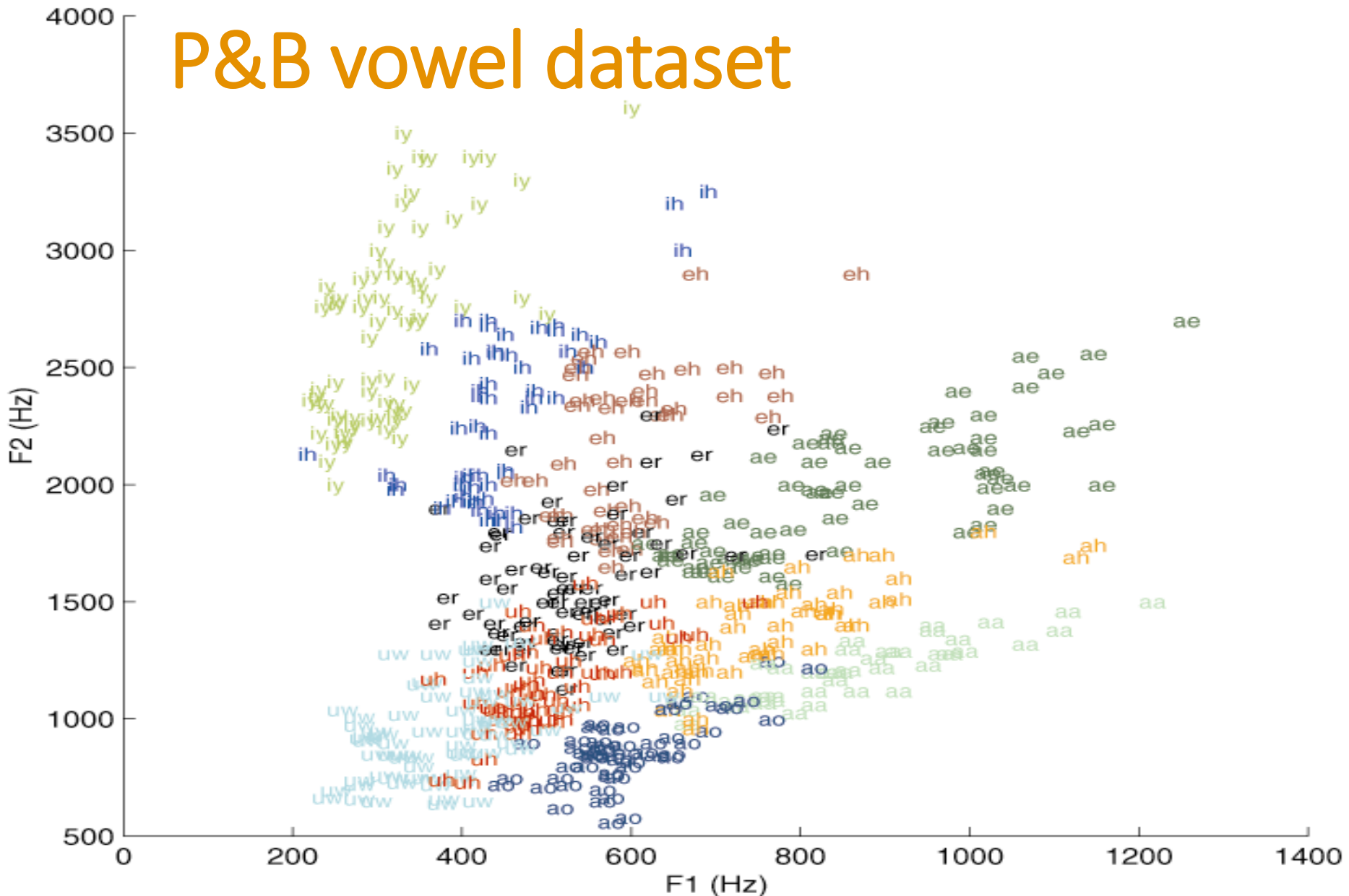


Decision regions

Examples with different classifiers using Peterson & Barney's vowel dataset

P&B vowel dataset

⇐ back “which part of the tongue is raised” front ⇒



⇐ close “how far the tongue is raised” open ⇒

Decision region for “KStar” classifier

